



UNIVERSITÀ  
DEGLI STUDI  
FIRENZE

## Sistema di Ticketing

Xiao Kang Luo  
7137112  
xiao.luo@edu.unifi.it

Silvia Zhu  
7055499  
silvia.zhu@edu.unifi.it

Progetto di Sistemi Operativi 2024/2025

26 January 2026

# INDICE

|          |                                             |          |
|----------|---------------------------------------------|----------|
| <b>1</b> | <b>Introduzione al progetto</b>             | <b>1</b> |
| 1.1      | Organizzazione del progetto                 | 1        |
| 1.2      | Compilazione                                | 2        |
| <b>2</b> | <b>Flusso di esecuzione e comunicazione</b> | <b>3</b> |
| 2.1      | Diagramma di flusso                         | 3        |
| 2.2      | Comunicazione                               | 4        |
| 2.3      | Input da tastiera                           | 6        |
| <b>3</b> | <b>Memoria e concorrenza</b>                | <b>7</b> |
| 3.1      | Strutture dati                              | 7        |
| 3.2      | Gestione della concorrenza                  | 8        |

# 1 Introduzione al progetto

In conformità con le richieste, l'architettura del sistema è stata suddivisa in due moduli principali: il `client` e il `server`. La comunicazione tra i due processi avviene tramite socket del dominio `AF_INET` di tipo `SOCK_STREAM` (protocollo `TCP`).

Dato che il sistema deve funzionare in un ambiente locale, l'indirizzo IP utilizzato per la connessione è `127.0.0.1` (`INADDR_LOOPBACK`) sulla porta `12345`.

## 1.1 Organizzazione del progetto

A fine di semplificare la gestione e la manutenzione del codice, il progetto è strutturato in più moduli distinti, ognuno con responsabilità specifiche.

Per il salvataggio di informazioni persistenti, si è optato per l'utilizzo di file in formato `json`. Questa scelta è motivata dalla necessità di avere una struttura dati gerarchica che rimanga leggibile e facilmente ispezionabile anche esternamente all'applicazione. Per il parsing e la manipolazione di tali file, è stata integrata la libreria open-source `cJSON` (<https://github.com/DaveGamble/cJSON>).

Allego di seguito una panoramica dell'organizzazione dei file del progetto:

```
ticketing_system/
├── bin/                # Contiene i file eseguibili
│   ├── client
│   └── server
├── build/             # Contiene i file oggetto
├── data/              # Contiene i file di dati in formato JSON
│   ├── tickets.json
│   └── users.json
├── include/           # Contiene i file intestazione
│   ├── client/
│   ├── json/
│   ├── server/
│   ├── ticket/
│   ├── user/
│   └── utils/
├── src/               # Contiene i file sorgente
│   ├── client/
│   ├── json/
│   ├── server/
│   ├── ticket/
│   ├── user/
│   └── utils/
└── ticketing_system.make
```

*Organizzazione dei file del progetto*

## 1.2 Compilazione

Per automatizzare e semplificare l'intero processo di build (compilazione e linking), è stato predisposto il file `ticketing_system.make`. Per generare gli eseguibili, è sufficiente posizionarsi nella directory principale ed eseguire il seguente comando:

```
$ make -f ticketing_system.make
```

*Compilazione del programma.*

I file eseguibili `client` e `server` verranno generati nella cartella `bin/`.

**ATTENZIONE:** Per il corretto avvio è necessario inserire i seguenti comandi (il server deve essere attivo prima del client) su 2 terminali separati DOPO essersi spostato nella root directory del progetto ( `ticketing_system/`, altrimenti i path relativi ai file non saranno corretti):

```
$ ./bin/server
```

*Esecuzione del server.*

La schermata del server dovrebbe apparire come segue:

```
Server ready (CTRL-C per terminare)
Waiting for a client connection...
```

*Schermata iniziale server*

Successivamente, si può avviare il client con il comando:

```
$ ./bin/client
```

*Esecuzione del client.*

A questo punto dovrebbe comparire la schermata iniziale sul terminale del client con le relative istruzioni per orientarsi:

```
[ISTRUZIONI]
Per selezionare un opzione del menu, inserire il numero corrispondente.
Per annullare l'inserimento in corso, digitare ESC + INVIO.
Premere INVIO per continuare...
```

*Schermata iniziale client*

**NOTA:** I file `tickets.json` e `users.json` presenti nella cartella `data/` contengono già dati per testare le funzionalità del sistema. Si consiglia di non modificarli manualmente per evitare inconsistenze.

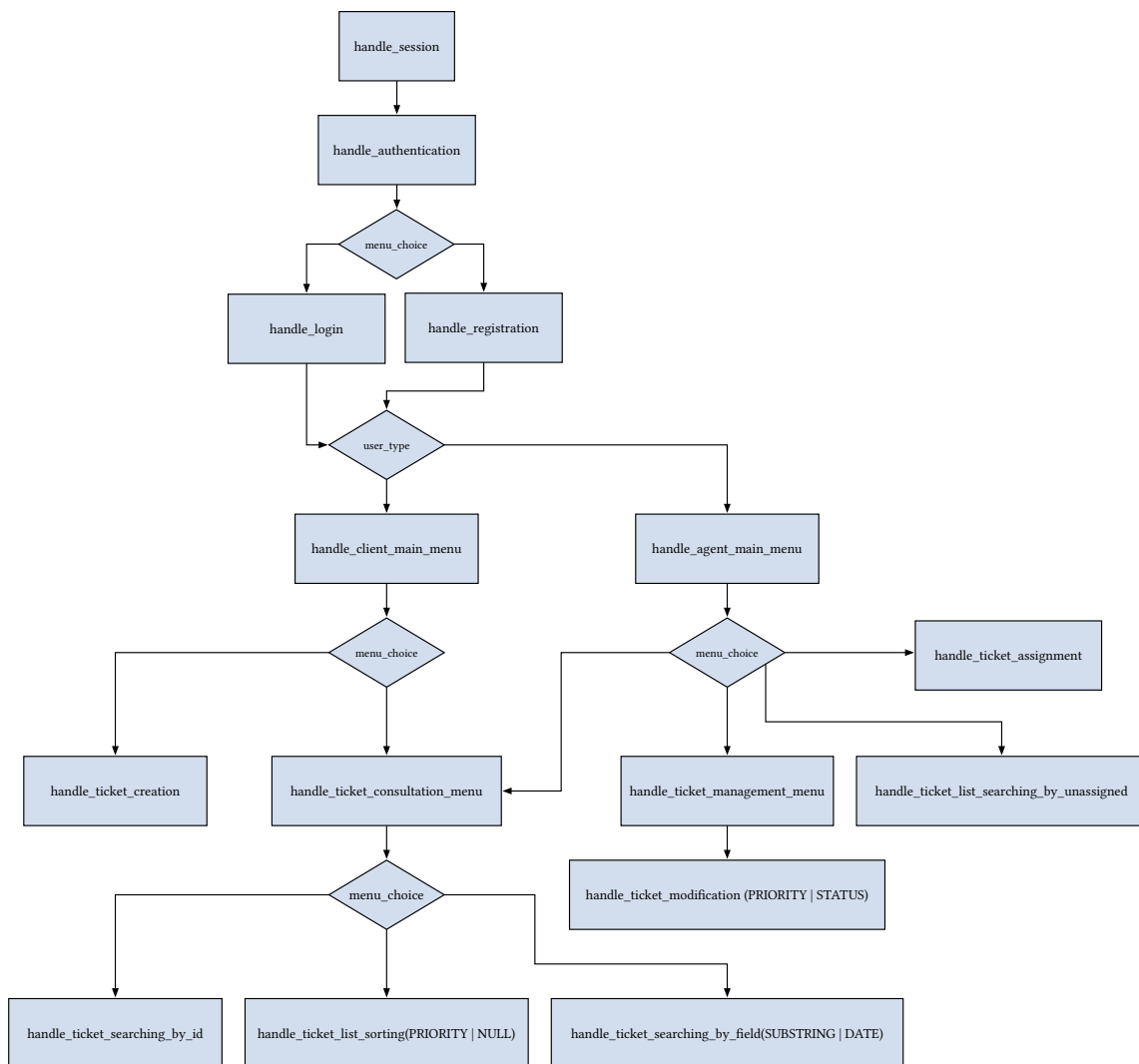
## 2 Flusso di esecuzione e comunicazione

### 2.1 Diagramma di flusso

Il `server` opera in modalità di ascolto continuo (con un limite di 5 client in coda di attesa) sulla socket designata. All'instaurazione di ogni nuova connessione, il processo padre invoca la chiamata di sistema `fork()`, generando un processo figlio dedicato.

Mentre il padre riprende immediatamente l'ascolto di nuove richieste, il processo figlio assume la gestione esclusiva dell'interazione con il client (tramite la funzione `handle_session`), occupandosi del suo intero ciclo di vita fino alla disconnessione.

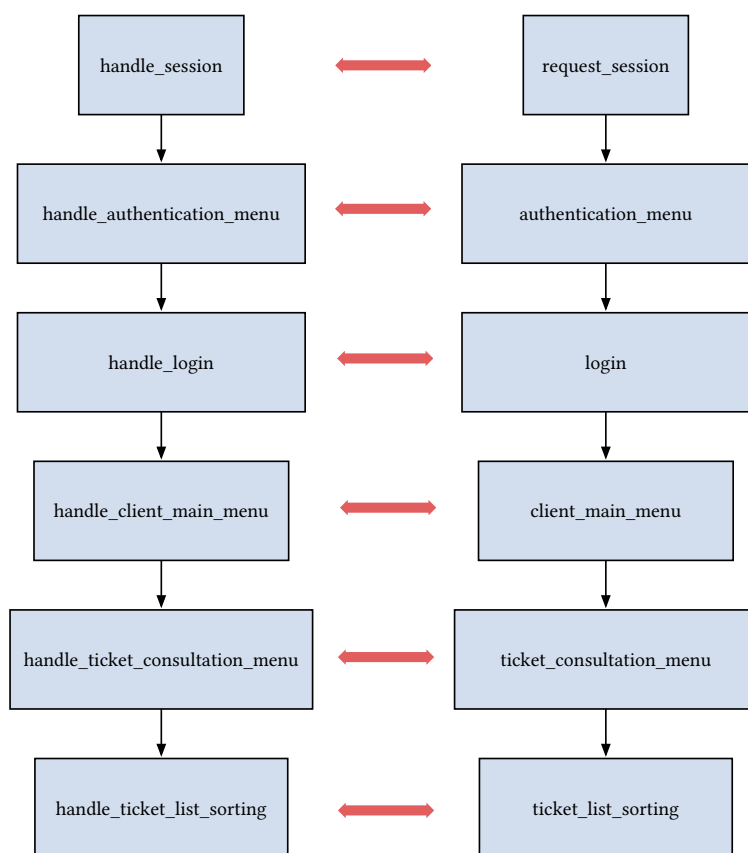
Il diagramma di flusso riportato nella seguente figura illustra la logica di funzionamento e la gestione delle connessioni dal lato server:



*Flowchart del funzionamento del server*

**NOTA:** Ad ogni blocco corrisponde una specifica funzione implementata nel codice sorgente, mentre il rombo indica un costrutto per il controllo del flusso ( `if-else` o `switch-case` ). I campi tra le parentesi rappresentano i valori secondo la quale è possibile parametrizzare la funzione: `handle_ticket_list_sorting(PRIORITY | NULL)` indica che la funzione può gestire una richiesta di ordinamento della lista dei ticket in base alla priorità o non effettuare alcun ordinamento.

Non mostriamo il diagramma di flusso del `client`, in quanto simmetrico a quello del `server`: anch'esso adotta una struttura basata su funzioni dedicate per ogni operazione richiesta dall'utente. Ci limitiamo tuttavia a fornire un esempio di interazione tra `client` (destra) e `server` (sinistra):



*Esempio di interazione tra client e server per la stampa ordinata dei ticket*

La corretta sincronizzazione tra `server` e `client` è garantita dalla comunicazione basata sulle chiamate di sistema `read()` e `write()` e la natura bloccante di queste ultime.

## 2.2 Comunicazione

Come già detto lo scambio di informazioni a basso livello avviene tramite `read()/write()` su socket `TCP`. Tuttavia, per garantire una corretta interpretazione dei dati trasmessi, è

stato definito un protocollo di comunicazione basato su specifici codici di stato (con gli `enum` in C) che fungono da segnali di controllo e coordinazione tra i due processi.

**CommunicationStatus** Definisce l'esito a basso livello delle operazioni di I/O sui socket (stato della rete).

- Valori: `COMM_SUCCESS`, `COMM_ERROR`, `COMM_CLOSED`.

**OperationOutcome** Rappresenta l'esito delle operazioni di business logic.

- Valori: `POSITIVE`, `NEGATIVE`, `ERROR`.

**DataAvailability** Funge da segnale preventivo (segnale di handshake) nel protocollo di comunicazione `client` → `server`. Indica se i dati richiesti sono pronti per la trasmissione o se l'operazione è stata annullata/invalidata. Il server procede a leggere i dati dal socket solo in caso di `DATA_AVAILABLE`.

- Valori: `DATA_UNAVAILABLE`, `DATA_AVAILABLE`.

Si aggiunge inoltre un quarto codice di stato, `InputOutcome`, utilizzato esclusivamente dal `Client` per gestire l'esito dell'input da tastiera dell'utente (lettura da `stdin`).

**InputOutcome** Codifica l'esito della lettura dell'input.

- Valori: `INPUT_SUCCESS`, `INPUT_ERROR`, `INPUT_EOF`, `INPUT_CANCEL`.

**Esempio** (lato client) - Funzione di login:

```
OperationOutcome login(...) {
    // Dichiarazione variabili
    DataAvailability availability;
    CommunicationStatus comm_status;
    OperationOutcome op_outcome;
    InputOutcome input_outcome;
    char username[USERNAME_BUFFER_SIZE];
    char password[PASSWORD_BUFFER_SIZE];

    // Raccolta credenziali da tastiera
    input_outcome = get_username_stdin(username);
    if (input_outcome == INPUT_SUCCESS) {
        input_outcome = get_password_stdin(password);
    }

    // Determinazione disponibilità dati
    availability = (input_outcome == INPUT_SUCCESS) ?
        DATA_AVAILABLE : DATA_UNAVAILABLE;

    // Invio disponibilità dati al server
    comm_status = send_availability(availability);
    if (comm_status != COMM_SUCCESS) {
        // Gestione errore comunicazione
        ...
    }
}
```

```

    }

    // Se i dati non sono disponibili, termina l'operazione
    if (availability != DATA_AVAILABLE) {return NEGATIVE;}

    // Invia username
    comm_status = send_string(socket, username);
    if(comm_status != COMM_SUCCESS) {...}

    // Invia password
    comm_status = send_string(socket, password);
    if(comm_status != COMM_SUCCESS) {...}

    // Riceve esito operazione
    comm_status = receive_outcome(socket, &op_outcome);
    if(...) {...}

    // Gestione esito comunicazione
    if(op_outcome == POSITIVE) {...}
    elseif (op_outcome == NEGATIVE) {...}
    else {...}

    return op_outcome;
}

```

### *Funzione di login (client)*

Per il trasferimento dei dati da presi da input, abbiamo quindi adottato un meccanismo di handshake preventivo: ancor prima dell'invio del payload (i dati effettivi, in questo caso sono le credenziali d'accesso), il `client` comunica al `server` se i dati sono pronti per essere trasmessi o se l'operazione è stata annullata (ad esempio, l'utente ha premuto `ESC + INVIO` durante l'inserimento). In tal caso, i dati per completare l'operazione sono incompleti e il `server` non tenterà di leggere nulla dal socket.

## 2.3 Input da tastiera

Per ridurre il carico sul `server` e minimizzare l'I/O su socket, il controllo di validità viene gestito con un approccio a due livelli:

Nel lato `client`, dopo l'invio della input, viene immediatamente verificato il rispetto dei vincoli formali (lunghezza minima, set di caratteri, formato). In caso di errore, l'invio è bloccato e all'utente è richiesto un reinserimento dei dati corretti.

Successivamente, se necessario, il `server` esegue i controlli di integrità e unicità (es. verificare che email o username non siano già registrati). In caso di esito negativo, l'operazione viene annullata e viene inviata una segnalazione di insuccesso al `client`, garantendo così la coerenza del database.



## 3 Memoria e concorrenza

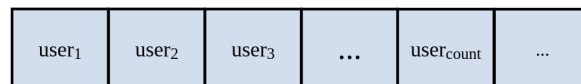
### 3.1 Strutture dati

Per accedere ai dati relativi a `user` e `ticket`, il `server` legge dai file json e carica in memoria principale due tipologie di strutture dati.

**UserList** Rappresenta una lista di utenti.

```
typedef struct UserList {  
    User *users;           // L'array dinamico  
    unsigned int count;    // contatore di utenti  
    unsigned int capacity; // capacità dell'array  
} UserList;
```

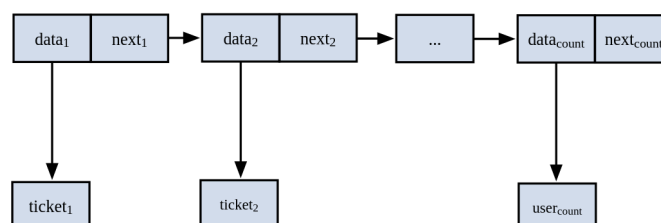
L'implementazione come array dinamico è stata scelta per massimizzare l'efficienza nelle operazioni di lettura (login), che risultano preponderanti rispetto alle scritture (registrazione). L'uso di un blocco di memoria contiguo permette di sfruttare la località spaziale. Il campo `capacity` gestisce la dimensione fisica allocata, permettendo al sistema di espandere l'array solo quando il numero effettivo di utenti ( `count` ) raggiunge tale soglia.



**TicketList** Rappresenta una lista di ticket.

```
typedef struct TicketList {  
    Node *head;           // Puntatore al primo nodo della lista  
    unsigned int count;    // Numero di elementi nella lista  
} TicketList;
```

L'implementazione come lista collegata (linked list) facilita le operazioni di inserimento, inoltre la necessità di ordinamento e filtro in base a criteri variabili rende la linked list una scelta più flessibile rispetto ad un array dinamico, che richiederebbe operazioni di riallocazione e copia più costose in termini di prestazioni.



## 3.2 Gestione della concorrenza

L'esecuzione concorrente di più processi `server` impone l'uso di meccanismi di mutua esclusione per l'accesso ai file. A tal fine, l'apertura dei file `tickets.json` e `users.json` avviene sempre mediante meccanismi di locking, applicando un `LOCK_SH` (shared) in fase di lettura e un `LOCK_EX` (exclusive) in fase di scrittura.

Poiché la chiamata a `fork()` genera processi con spazi di memoria separati, le strutture dati non sono condivise: i file diventano l'unico mezzo di condivisione di dati tra processi. Ogni processo che desidera accedere ai dati deve necessariamente effettuare un'operazione di `load` (seguita da una `save` al termine della modifica).

**Esempio** (lato server) - Funzione di creazione ticket:

```
OperationOutcome handle_ticket_creation(...) {  
    ...  
    // Creazione ticket con dati forniti dal client  
    Ticket new_ticket = create_ticket(...);  
    ...  
    // Caricamento dati da file (scrittura)  
    // Apertura file con LOCK_EX  
    TicketList tickets = load_ticket_list("r+");  
    ...  
    // Aggiornamento struttura dati in memoria  
    add_ticket(tickets, new_ticket);  
    ...  
    // Salvataggio dati su file  
    // Chiusura file con unlock  
    save_ticket_list(tickets);  
    ...  
    // Ritorna esito operazione  
    return ...;  
}
```

*handle\_ticket\_creation (server)*

Come già accennato, per massimizzare l'efficienza, il server accede al file su disco solo dopo aver raccolto tutti i dati necessari (es. le informazioni per la creazione del ticket), minimizzando così la durata della sezione critica e il conseguente blocco degli altri processi.

Tuttavia, l'obbligo di effettuare il caricamento e il salvataggio completo delle strutture ad ogni singola operazione introduce un notevole overhead di I/O. Questo rende l'implementazione poco scalabile e inefficiente a fronte di grandi volumi di dati.